Document 2: Phase-Wise Backend Execution & Master Prompts Guide

---

Executive Guidance for Backend Lead (Person A)

This document provides a phase-by-phase implementation roadmap for developing the backend of the **Smart Learning & AI Media OS**. You will execute this roadmap in **Antigravity IDE** or using **Claude Code**. Each phase includes:

1. **Target Execution Directories:** Explicit file paths inside `apps/api/src/`.

2. **Step-by-Step Procedure:** Specific instructions detailing what to write and how components connect.

3. **Master Prompt for AI Agent:** A self-contained, context-rich prompt ready to feed into Antigravity to generate clean TypeScript code.

PHASE 0: Infrastructure, Monorepo & Environment Configuration

* **Timeline:** Days 1–5 (Phase 0)

* **Primary Objective:** Initialize the monorepo workspace, configure `apps/api` with TypeScript, validate environment variables using Zod, connect to MongoDB Atlas, and run Redis locally.

* **Target Directories:** `apps/api/src/config/`, `apps/api/src/index.ts`, `pnpm-workspace.yaml`, `Dockerfile`.

Step-by-Step Backend Procedure

1. **Monorepo Setup:** Create root `pnpm-workspace.yaml` declaring `apps/*` and `packages/*`. Initialize `apps/api/package.json` with dependencies (`express`, `mongoose`, `dotenv`, `cors`, `helmet`, `bcryptjs`, `jsonwebtoken`, `zod`, `bullmq`, `ioredis`).

2. **Environment Validation (`apps/api/src/config/env.ts`):** Define a Zod schema validating process environment variables (`PORT`, `MONGODB_URI`, `JWT_SECRET`, `JWT_REFRESH_SECRET`, `REDIS_URL`, `CLOUDINARY_URL`, `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`). Throw a runtime error if any key is missing.

3. **Database Connection (`apps/api/src/config/db.ts`):** Establish a Mongoose connection to MongoDB Atlas with re-connection handling.

4. **Server Entry Point (`apps/api/src/index.ts`):** Instantiate Express with `helmet`, `cors`, `express.json()`, and a `GET /health` health-check route.

5. **Local Redis Environment:** Launch Redis using Docker (`docker run -d -p 6379:6379 redis:alpine`) for local queue testing.

Master Prompt for Phase 0 (Antigravity / Claude Code)

```
SYSTEM ROLE: You are the Lead Backend Architect setup bot.

TASK: Initialize the Backend Express + TypeScript framework for "apps/api" within a pnpm mo
norepo structure.

REQUIREMENTS:

1. Initialize package.json in "apps/api" with dependencies: express, mongoose, dotenv, cors
, helmet, bcryptjs, jsonwebtoken, zod, bullmq, ioredis, @types/node, typescript, ts-node-de
v.

2. Create tsconfig.json tuned for Node 20+ with ES2022 output, modules resolution, and stri
ct type checking enabled.

3. Create "src/config/env.ts" using Zod to parse process.env (PORT, MONGODB_URI, JWT_SECRET
, JWT_REFRESH_SECRET, REDIS_URL). Throw error if any variable is missing.

4. Create "src/config/db.ts" establishing connection to MongoDB Atlas using Mongoose.

5. Create "src/index.ts" configuring Express server with helmet, CORS, JSON body parsing, g
lobal error handling middleware, and GET /health returning JSON status.

OUTPUT FORMAT: Provide the clean directory structure and complete, runnable file contents w
ithout placeholders.
```

PHASE 1: Authentication, Dual Workspace & Core Schemas

* **Timeline:** Weeks 1–3 (Phase 1)

* **Primary Objective:** Build user registration, authentication with access/refresh token rotation, role-based authorization, and dual-workspace management.

* **Target Directories:** `apps/api/src/models/`, `apps/api/src/routes/`, `apps/api/src/middleware/`, `apps/api/src/services/auth/`.

Step-by-Step Backend Procedure

1. **Mongoose Schemas:**

* `User.ts`: `fullName`, `email`, `passwordHash`, `role` (`student`, `instructor`, `org_admin`), `personalWorkspaceId`, `currentOrgId`.

* `Workspace.ts`: `name`, `type` (`PERSONAL`, `ORGANIZATIONAL`), `ownerId`, `organizationDomain`, `members` array.

* `Organization.ts`: `name`, `domain`, `ownerId`.

2. **Authentication Controllers (`apps/api/src/routes/auth.routes.ts`):**

* `POST /api/v1/auth/register`: Hashes passwords via `bcryptjs`, creates the `User` document, automatically provisions an isolated `Private Workspace`, and returns a JWT access token alongside an `httpOnly` refresh token cookie.

* `POST /api/v1/auth/login`: Validates credentials and returns updated JWT tokens.

* `POST /api/v1/auth/refresh`: Rotates short-lived access tokens using the valid HTTP-only refresh cookie.

3. **Authorization Middleware (`apps/api/src/middleware/auth.ts`):**

* `requireAuth`: Extracts and verifies Bearer tokens from incoming HTTP headers.

* `requireRole`: Restricts access based on allowed user roles (e.g., `org_admin`).

4. **Workspace Controller (`apps/api/src/routes/workspace.routes.ts`):**

* `POST /api/v1/workspaces/switch`: Updates `currentOrgId` on the active user session to switch between Personal and Organizational contexts.

Master Prompt for Phase 1 (Antigravity / Claude Code)

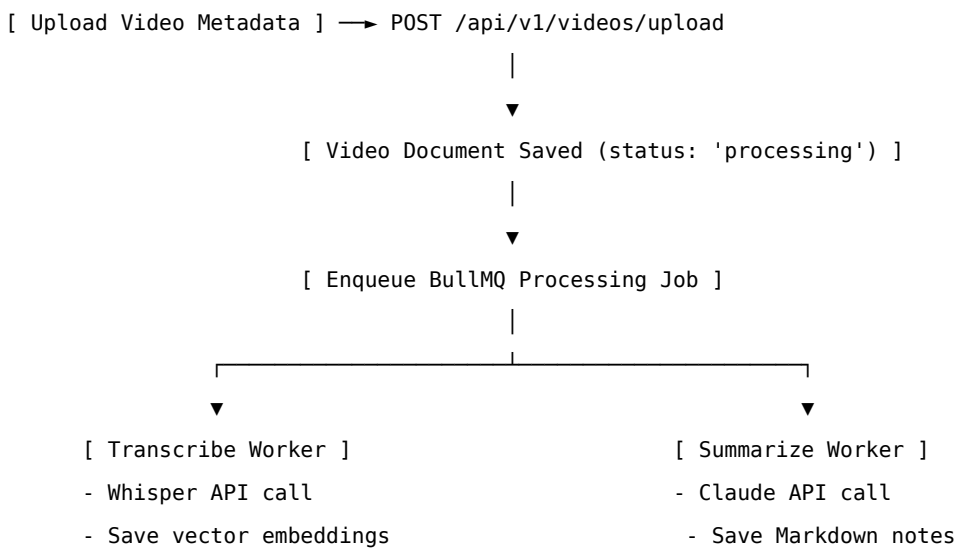
```
SYSTEM ROLE: You are the Lead Backend Engineer implementing Auth & Workspace Architecture.

TASK: Build complete Authentication and Dual-Workspace REST API in Express + TypeScript + M
ongoose.

REQUIREMENTS:

1. Schemas in "src/models/":

   - UserSchema: fullName, email, passwordHash, role ('student'|'instructor'|'org_admin'),
personalWorkspaceId, currentOrgId.

   - WorkspaceSchema: name, type ('PERSONAL'|'ORGANIZATIONAL'), ownerId, organizationDomain
, members array [{ userId, role }].

   - OrganizationSchema: name, domain, ownerId.

2. Auth Logic in "src/services/auth.service.ts" & "src/routes/auth.routes.ts":

   - Register route: Hashes password with bcrypt, creates User, auto-creates their Personal
 Workspace, returns JWT access token + httpOnly refresh cookie.

   - Login route: Validates password, returns access token + sets refresh token cookie.

   - Refresh route: Verifies refresh token and issues a new access token.

3. Middleware in "src/middleware/auth.ts":

   - "requireAuth": Extracts Bearer JWT token from Authorization header and attaches decode
d payload to req.user.

   - "requireRole(roles)": Ensures req.user.role matches allowed scopes.

4. Workspace Logic in "src/routes/workspace.routes.ts":

   - POST /api/v1/workspaces/switch: Accepts { workspaceId } and updates req.user.currentOr
gId accordingly.

OUTPUT FORMAT: Generate complete, fully-typed TypeScript code for models, routes, controlle
rs, and middleware.
```

PHASE 2: Core Learning Loop – Video Pipeline, Async Queue & Code Execution

* **Timeline:** Weeks 4–10 (Phase 2)

* **Primary Objective:** Build the video streaming API, construct background queues for Whisper transcription and Claude summarization, set up the Judge0 code execution proxy, and implement RAG Doubt Assistant lookups.

* **Target Directories:** `apps/api/src/workers/`, `apps/api/src/routes/`, `apps/api/src/services/ai/`.



Step-by-Step Backend Procedure

1. **Video Endpoint (`apps/api/src/routes/video.routes.ts`):**

* `POST /api/v1/videos/upload`: Ingests video metadata from Cloudinary, creates a `Video` document (`status: 'processing'`), and enqueues a `processVideo` job.

2. **Background Queue Engine (`apps/api/src/workers/videoWorker.ts`):**

* Configures a BullMQ worker backed by ioredis.

* **Step 1:** Sends video audio track to OpenAI Whisper API for timestamped transcript extraction.

* **Step 2:** Converts transcript chunks to 1536-dimensional embeddings (`text-embedding-3-small`) and saves them to `TranscriptSegment`.

* **Step 3:** Sends transcript text to Claude API to generate Markdown summary notes and JSON chapter markers.

* **Step 4:** Updates `Video` status to `ready`.

3. **Judge0 Code Sandbox Proxy (`apps/api/src/routes/code.routes.ts`):**

* `POST /api/v1/code/execute`: Receives `{ language, sourceCode, stdin }` and proxies to Judge0 with a 5-second execution limit and 128MB RAM ceiling.

4. **RAG Doubt Assistant (`apps/api/src/routes/ai.routes.ts`):**

* `POST /api/v1/videos/:id/doubt`: Converts student questions into vector embeddings, runs `$vectorSearch` in MongoDB Atlas, retrieves matching context, and streams grounded answers from Claude API.

Master Prompt for Phase 2 (Antigravity / Claude Code)

```
SYSTEM ROLE: You are the Senior AI & Media Infrastructure Backend Lead.

TASK: Build the core Video Processing Queue, Judge0 Code Execution Proxy, and RAG Doubt Ass
istant endpoints.

REQUIREMENTS:

1. Video Queue & Worker ("src/workers/videoWorker.ts"):

   - Set up BullMQ queue "video-processing" using IORedis connection.

   - Define processVideo worker job:

     a. Calls OpenAI Whisper API to get transcript text with timestamps.

     b. Generates vector embeddings (1536 dims) for transcript chunks and saves to "Transcr
iptSegment" schema.

     c. Calls Anthropic Claude API to generate structured Markdown notes and dynamic JSON c
hapter markers.

     d. Updates Video model status to "ready".

2. Judge0 Proxy Route ("src/routes/code.routes.ts"):

   - POST /api/v1/code/execute: Accepts { language_id, source_code, stdin }.

   - Posts request to Judge0 execution engine URL with CPU execution limit = 5s and memory
cap = 128MB.

   - Returns { stdout, stderr, compile_output, time, memory }.

3. RAG Doubt Assistant Route ("src/routes/ai.routes.ts"):

   - POST /api/v1/videos/:id/doubt: Embeds student query string.

   - Executes MongoDB Atlas $vectorSearch pipeline against "transcriptsegments" collection
filtered by videoId.

   - Passes retrieved text context + question to Claude API with system prompt forcing resp
onse grounded ONLY in context.

OUTPUT FORMAT: Provide complete worker, model, and controller files with complete typed err
or handling.
```

PHASE 3: Practice & Career AI Modules Backend

* **Timeline:** Weeks 11–18 (Phase 3)

* **Primary Objective:** Build services for JAM (Just A Minute) speech evaluation, company & role-specific resume tailoring, turn-based mock interviews, and skill gap calculation.

* **Target Directories:** `apps/api/src/routes/`, `apps/api/src/services/ai/`, `apps/api/src/models/`.

Step-by-Step Backend Procedure

1. **JAM Session Speech Evaluator (`apps/api/src/routes/jam.routes.ts`):**

* `POST /api/v1/jam-sessions/evaluate`: Accepts uploaded WebRTC audio recordings, calls Whisper API for transcription, computes Words Per Minute (WPM) and filler word counts (`"um"`, `"ah"`, `"like"`), evaluates technical accuracy via Claude API, and stores a `JamSession` record.

2. **Company & Role Resume Tailoring (`apps/api/src/routes/resume.routes.ts`):**

* `POST /api/v1/resumes/tailor`: Accepts raw resume content alongside target company and target role parameters. Instructs Claude API to rewrite experience bullet points into the Google XYZ format (*Accomplished X as measured by Y by doing Z*), calculates ATS match percentage, and identifies missing skills.

3. **Turn-Based Mock Interview Engine (`apps/api/src/routes/interview.routes.ts`):**

* `POST /api/v1/interviews/start` & `POST /api/v1/interviews/answer`: Manages adaptive turn-based technical interview questions, grading candidate answers dynamically using Claude API.

Master Prompt for Phase 3 (Antigravity / Claude Code)

```
SYSTEM ROLE: You are the Lead Backend Engineer for AI Career Modules.

TASK: Build API routes and services for JAM Speech Evaluation, Targeted Resume Tailoring, a
nd AI Mock Interviews.

REQUIREMENTS:

1. JAM Session Endpoint ("src/routes/jam.routes.ts"):

   - POST /api/v1/jam-sessions/evaluate: Multi-part form upload receiving audio file + topi
cPrompt.

   - Transcribes audio via Whisper API.

   - Calculates WPM = (total words / audio duration in minutes).

   - Regex counts filler words ("um", "uh", "like", "you know", "actually").

   - Prompts Claude API to evaluate grammar and technical accuracy (1-100 score).

   - Persists record in JamSession model and returns full evaluation card JSON.

2. Resume Tailoring Endpoint ("src/routes/resume.routes.ts"):

   - POST /api/v1/resumes/tailor: Accepts { rawResumeText, targetCompany, targetRole }.

   - Instructs Claude API to rewrite experience bullets using Google XYZ format customized
for targetCompany/Role.

   - Calculates keyword match percentage (ATS score 1-100) and returns missing keyword reco
mmendations.

3. Mock Interview Router ("src/routes/interview.routes.ts"):

   - POST /api/v1/interviews/start: Returns initial technical question based on role.

   - POST /api/v1/interviews/answer: Accepts user answer text, evaluates correctness with C
laude API, returns immediate score feedback, and provides next question.

OUTPUT FORMAT: Generate production-ready Express routes, Zod request schemas, and Claude pr
ompt integration logic.
```

PHASE 4: Real-Time Collaboration & Knowledge Infrastructure

* **Timeline:** Weeks 19–25 (Phase 4)

* **Primary Objective:** Build LiveKit WebRTC room token minting, Socket.io real-time whiteboard state synchronization, global semantic video search, and an SM-2 flashcard review scheduler.

* **Target Directories:** `apps/api/src/routes/`, `apps/api/src/gateways/`, `apps/api/src/services/`.

Step-by-Step Backend Procedure

1. **LiveKit Token Service (`apps/api/src/routes/livekit.routes.ts`):**

* `POST /api/v1/study-rooms/token`: Mints signed LiveKit WebRTC join tokens for group study video sessions.

2. **Socket.io Real-Time Gateway (`apps/api/src/gateways/studyRoom.gateway.ts`):**

* Manages WebSocket events for active study rooms, syncing cursor locations, chat messages, and shared whiteboard canvas drawing states.

3. **Global Semantic Search (`apps/api/src/routes/search.routes.ts`):**

* `POST /api/v1/search`: Embeds search queries, executes vector similarity searches across accessible transcripts, and returns timestamped matches.

4. **Flashcard SM-2 Engine (`apps/api/src/routes/flashcard.routes.ts`):**

* Implements the SuperMemo-2 (SM-2) algorithm calculating repetition intervals based on student review ratings (`0–5` scale).

* `GET /api/v1/flashcards/due`: Retrieves cards scheduled for review on the current date.

Master Prompt for Phase 4 (Antigravity / Claude Code)

```
SYSTEM ROLE: You are the Real-Time & Systems Backend Specialist.

TASK: Implement LiveKit token minting, Socket.io study room gateway, global Semantic Search
, and SM-2 Flashcard review scheduler.

REQUIREMENTS:

1. LiveKit Token Controller ("src/routes/livekit.routes.ts"):

   - POST /api/v1/study-rooms/token: Receives { roomName, identity }, uses 'livekit-server-
sdk' to generate a token with video join grants, and returns token string.

2. Socket.io Gateway ("src/gateways/studyRoom.gateway.ts"):

   - Handle connection, room join/leave events, cursor movement broadcasts, and whiteboard
drawing state sync ("draw-event").

3. Global Semantic Search ("src/routes/search.routes.ts"):

   - POST /api/v1/search: Accepts { queryText }, embeds query string via OpenAI embeddings
API.

   - Runs MongoDB Atlas $vectorSearch on TranscriptSegment collection filtered by user's ac
cessible workspace ID.

   - Returns sorted matching segments with video metadata and exact timestamp seconds.

4. SM-2 Flashcard Engine ("src/routes/flashcard.routes.ts"):

   - POST /api/v1/flashcards/:id/review: Accepts { rating } (0-5 scale).

   - Applies SuperMemo-2 algorithm to update repetitionCount, easeFactor, interval, and nex
tReviewDate.

   - GET /api/v1/flashcards/due: Queries flashcards where nextReviewDate <= current date.

OUTPUT FORMAT: Provide complete TypeScript source code with correct algorithmic math for SM
-2 and clean Socket.io handlers.
```

PHASE 5: Cohort Analytics, Certificates & Offline Engine

* **Timeline:** Weeks 26–30 (Phase 5)

* **Primary Objective:** Build aggregate cohort analytics queries, QR-coded PDF completion certificate generation, and verification routes.

* **Target Directories:** `apps/api/src/routes/`, `apps/api/src/services/`.

Step-by-Step Backend Procedure

1. **Cohort Analytics (`apps/api/src/routes/analytics.routes.ts`):**

* `GET /api/v1/orgs/:id/analytics`: Executes MongoDB aggregation pipelines calculating average course completion rates, mean quiz scores, platform usage heatmaps, and cohort skill gaps.

2. **Cryptographic Certificate Generator (`apps/api/src/routes/certificate.routes.ts`):**

* `POST /api/v1/certificates/generate`: Validates course completion, compiles a styled PDF certificate using `pdf-lib`, embeds a dynamic QR code pointing to `/verify/:id`,

uploads the PDF to Cloudinary, and saves a `Certificate` record.

* `GET /api/v1/certificates/verify/:id`: Public endpoint returning certificate authenticity metadata.

Master Prompt for Phase 5 (Antigravity / Claude Code)

```
SYSTEM ROLE: You are the Enterprise Analytics & PDF Engineering Backend Lead.

TASK: Build cohort analytics aggregation endpoints, QR-coded PDF certificate generation, an
d verification routes.

REQUIREMENTS:

1. Cohort Analytics ("src/routes/analytics.routes.ts"):

   - GET /api/v1/orgs/:id/analytics: Guarded by requireRole(['org_admin', 'instructor']).

   - Runs Mongoose aggregation pipeline computing:

     a. Average course completion % across all workspace members.

     b. Mean quiz score across cohorts.

     c. Heatmap of daily platform activity.

2. Certificate Generator ("src/routes/certificate.routes.ts"):

   - POST /api/v1/certificates/generate: Takes { courseId, userId }.

   - Verifies 100% video completion & passing quiz score threshold.

   - Uses 'pdf-lib' and 'qrcode' packages to generate a styled PDF certificate containing s
tudent name, course title, issue date, and dynamic QR verification code.

   - Uploads PDF to Cloudinary and saves Certificate model record.

   - GET /api/v1/certificates/verify/:id: Public verification endpoint returning certificat
e metadata if valid.

OUTPUT FORMAT: Provide clean, working Mongoose aggregation pipelines and PDF rendering logi
c.
```

PHASE 6: Hardening, Rate Limiting & Production Deployment

* **Timeline:** Weeks 31–34 (Phase 6)

* **Primary Objective:** Implement strict API rate limiting, audit execution sandbox safety, write multi-stage Docker build files, and configure Sentry error tracking.

* **Target Directories:** `apps/api/Dockerfile`, `apps/api/src/middleware/`.

Step-by-Step Backend Procedure

1. **API Rate Limiter (`apps/api/src/middleware/rateLimiter.ts`):** Configure Upstash Redis rate limiters: AI routes (10 requests/min), code execution (20 requests/min),

general API (100 requests/min).

2. **Sandbox Audit:** Verify Judge0 execution containers run without network access, under non-root users, capped at 128MB RAM and 5 seconds CPU time.

3. **Multi-Stage Docker Setup (`apps/api/Dockerfile`):** Build TypeScript assets in Stage 1 and copy `/dist` to a lightweight Node.js 20 Alpine runner image in Stage 2.

4. **Sentry Error Handling (`apps/api/src/middleware/error.ts`):** Capture unhandled exceptions and send error reports to Sentry while returning sanitized error payloads to users.

Master Prompt for Phase 6 (Antigravity / Claude Code)

```
SYSTEM ROLE: You are the Lead DevOps & Security Backend Architect.

TASK: Finalize production Docker setup, enforce strict API rate limiting, add Sentry error
tracking, and secure endpoints for launch.

REQUIREMENTS:

1. Docker Configuration ("Dockerfile" in apps/api):

   - Multi-stage build for Node.js 20 LTS.

   - Stage 1: Build TypeScript into /dist directory using pnpm.

   - Stage 2: Minimal alpine runner image installing production dependencies only.

   - Expose PORT 5000 and set CMD ["node", "dist/index.js"].

2. Rate Limiter Middleware ("src/middleware/rateLimiter.ts"):

   - Use "@upstash/ratelimit" and "@upstash/redis".

   - Define custom rate limiters:

     a. aiLimiter: 10 requests per 1 min.

     b. codeExecLimiter: 20 requests per 1 min.

     c. generalLimiter: 100 requests per 1 min.

3. Global Error Handler ("src/middleware/error.ts"):

   - Catches unhandled errors, logs exception context to Sentry (@sentry/node), and returns
 clean sanitized JSON error responses without stack traces in production.

OUTPUT FORMAT: Provide complete Dockerfile, rate limiting middleware, and production-ready
Express error handler setup.
```